

TP INITIATION RASPBERRY PI4

Systèmes Embarqués et IoT

Durée : 3 heures • Niveau : Initiation

Classe : _____

Enseignant : Korota Arsène COULIBALY



20 août 2026

1. Objectifs pédagogiques

À l'issue de ce TP, vous serez capables de :

- Configurer un Raspberry Pi 4 (mise à jour système, activation SSH/VNC).
- Utiliser les commandes Linux de base dans un contexte système embarqué.
- Se connecter au Raspberry Pi via SSH et VNC depuis un poste distant.
- Câbler et programmer une LED sur les ports GPIO.
- Créer un programme Python pour faire clignoter une LED.
- Publier l'état d'une LED vers un broker MQTT avec Python.

2. Matériel nécessaire

2.1 Matériel obligatoire

- 1 Raspberry Pi 4 (2 Go de RAM minimum).
- 1 carte microSD (16 Go minimum, classe 10).
- 1 alimentation officielle Raspberry Pi (5V, 3A, USB-C).
- 1 câble Ethernet RJ45.
- 1 LED (rouge ou verte).
- 1 résistance 220 Ω .
- 2 câbles Dupont femelle-mâle.
- 1 plaque d'essai (breadboard) — facultatif.

2.2 Logiciels nécessaires

- Raspberry Pi OS (64-bit) installé sur la carte SD.
- Poste client avec :
 - Client SSH (PuTTY sous Windows, ou terminal sous Linux/Mac).
 - Visionneuse VNC (RealVNC Viewer recommandé).
 - Broker MQTT (Mosquitto installé localement ou broker public).

3. Préparation du Raspberry Pi

3.1 Premier démarrage

1. Insérer la carte microSD contenant Raspberry Pi OS dans le Raspberry Pi.
2. Connecter le câble Ethernet au Raspberry Pi et au réseau local.
3. Connecter un clavier, une souris et un écran HDMI (pour la configuration initiale).
4. Brancher l'alimentation USB-C.

Le système démarre automatiquement et affiche le bureau Raspbian.

3.2 Mise à jour du système

Ouvrir un terminal et saisir :

```
1 sudo apt update
2 sudo apt full-upgrade -y
3 sudo reboot
```

Explication :

- `sudo` : exécute la commande avec les droits administrateur.
- `apt update` : met à jour la liste des paquets disponibles.
- `apt full-upgrade` : installe les mises à jour système.
- `reboot` : redémarre le Raspberry Pi.

Temps estimé : 5–15 minutes selon la connexion Internet.

4. Commandes Linux essentielles

4.1 Navigation dans le système de fichiers

Après le redémarrage, ouvrir un terminal et pratiquer les commandes suivantes :

Commande	Description	Exemple
<code>pwd</code>	Affiche le répertoire courant	<code>pwd</code>
<code>ls</code>	Liste les fichiers du répertoire	<code>ls -al</code>
<code>cd</code>	Change de répertoire	<code>cd Documents</code>
<code>cd ..</code>	Remonte d'un niveau	<code>cd ..</code>
<code>mkdir</code>	Crée un répertoire	<code>mkdir tp_iot</code>
<code>rmdir</code>	Supprime un répertoire vide	<code>rmdir test</code>

Exercice pratique 1 :

```
1 cd ~
2 mkdir tp_iot
3 cd tp_iot
4 pwd
5 ls -al
```

Question 1 : Quel est le chemin absolu de votre répertoire `tp_iot` ?

4.2 Manipulation de fichiers

Commande	Description	Exemple
<code>touch</code>	Crée un fichier vide	<code>touch test.txt</code>
<code>nano</code>	Éditeur de texte simple	<code>nano test.txt</code>
<code>cat</code>	Affiche le contenu d'un fichier	<code>cat test.txt</code>
<code>cp</code>	Copie un fichier	<code>cp test.txt test2.txt</code>
<code>mv</code>	Déplace/renomme un fichier	<code>mv test.txt nouveau.txt</code>
<code>rm</code>	Supprime un fichier	<code>rm -i test2.txt</code>

Exercice pratique 2 :

```
1 cd ~/tp_iot
2 echo "Bonjour Raspberry Pi" > message.txt
3 cat message.txt
4 cp message.txt sauvegarde.txt
5 ls -l
```

4.3 Commandes système et réseau

Commande	Description	Exemple
ip addr	Affiche les adresses IP	ip addr
hostname -I	Affiche l'adresse IP locale	hostname -I
ping	Teste la connectivité réseau	ping -c 4 8.8.8.8
df -h	Affiche l'espace disque	df -h
free -h	Affiche la mémoire disponible	free -h
systemctl status	État d'un service	systemctl status ssh

Exercice pratique 3 :

```
1 hostname -I
```

Question 2 : Notez l'adresse IP de votre Raspberry Pi (utile pour SSH/VNC).

4.4 Gestion des permissions

Commande	Description	Exemple
chmod +x	Rend un fichier exécutable	chmod +x script.py
chmod 755	Définit les permissions	chmod 755 script.py
chown	Change le propriétaire	sudo chown pi:pi fichier

5. Configuration SSH

SSH (Secure Shell) permet de se connecter au Raspberry Pi à distance via le terminal.

5.1 Activation de SSH

```
1 sudo raspi-config
```

Puis via l'interface :

- Sélectionner **3 Interface Options**.
- Sélectionner **I2 SSH**.
- Sélectionner **Yes** pour activer SSH.
- Confirmer avec **Ok**, puis **Finish**.

5.2 Connexion SSH depuis un poste client

Sous Windows (PuTTY) :

1. Lancer PuTTY et saisir l'adresse IP du Raspberry Pi dans *Host Name*.
2. Port : 22 ; Connection type : SSH puis **Open**.
3. Accepter la clé à la première connexion ; **Login** : `pi`, mot de passe : `raspberrypi` (par défaut).

Sous Linux/Mac :

```
1 ssh pi@[ADRESSE_IP_DU_PI]
2 # Exemple :
3 ssh pi@192.168.1.50
```

Exercice pratique 4 :

```
1 uname -a
2 cat /proc/cpuinfo | grep Model
```

Question 3 : Quelle version du Raspberry Pi utilisez-vous ?

6. Configuration VNC

VNC (Virtual Network Computing) permet d'accéder au bureau graphique du Raspberry Pi à distance.

6.1 Activation de VNC

Via SSH ou terminal local :

```
1 sudo raspi-config
```

Puis :

— **3 Interface Options** → **I3 VNC** → **Yes**.

6.2 Connexion VNC

1. Installer **RealVNC Viewer** sur le poste client.
2. Saisir l'IP du Raspberry Pi dans la barre d'adresse.
3. Accepter le certificat puis s'authentifier : utilisateur `pi`, mot de passe `raspberrypi`.

Exercice pratique 5 : via VNC, ouvrir Chromium et vérifier l'accès Internet.

7. Câblage d'une LED sur GPIO

7.1 Comprendre les GPIO

Le Raspberry Pi 4 dispose de 40 broches GPIO. Utiliser la numérotation **BCM**.

Brochage utile :

- **GPIO 17** (broche physique 11) : LED.
- **GND** (broches 6, 9, 14, 20, 25, 30, 34, 39) : masse.

Visualiser le brochage :

```
1 pinout
```

7.2 Schéma de câblage

```
1 GPIO 17 (broche 11) -> R sistance 220    -> Anode LED (+, patte
   longue)
2 Cathode LED (-, patte courte) -> GND (broche 6)
```

Important :

- L'anode (+) est la patte **longue**.
- La cathode (-) est la patte **courte**.
- La résistance limite le courant dans la LED.

7.3 Réalisation du montage

1. **Raspberry Pi éteint**, réaliser le câblage décrit.
2. Allumer le Raspberry Pi.

Attention : ne pas manipuler les broches sous tension.

8. Programme Python : LED clignotante

8.1 Installation de la bibliothèque GPIO Zero

```
1 sudo apt install -y python3-gpiozero
```

8.2 Création du programme

```
1 cd ~/tp_iot
2 nano led_blink.py
```

Saisir :

```
1 #!/usr/bin/env python3
2 from gpiozero import LED
3 from time import sleep
4
5 # Configuration
6 LED_GPIO = 17    # Num ro BCM du GPIO
7 INTERVALLE = 0.5 # secondes
8
9 # Initialisation de la LED
10 led = LED(LED_GPIO)
11
12 print("D marrage du clignotement de la LED sur GPIO", LED_GPIO)
13 print("Appuyez sur Ctrl+C pour arr ter")
```

```

14
15 try:
16     while True:
17         led.on()
18         print("LED allum e")
19         sleep(INTERVALLE)
20         led.off()
21         print("LED teinte ")
22         sleep(INTERVALLE)
23 except KeyboardInterrupt:
24     print("\nArr t du programme")
25     led.off()
26     print("LED teinte , GPIO lib r ")

```

Rendre exécutable et lancer :

```

1 chmod +x led_blink.py
2 python3 led_blink.py

```

Question 4 : que se passe-t-il avec `INTERVALLE = 0.1` ?

9. Programme Python : publication MQTT

9.1 Objectif

Créer un programme qui fait clignoter la LED et publie son état (ON/OFF) vers un broker MQTT.

9.2 Installation de Paho MQTT

```

1 python3 -m pip install --user paho-mqtt

```

9.3 Configuration du broker MQTT

Option 1 : broker public (test) — `test.mosquitto.org:1883`, sans authentification.

Option 2 : broker local :

```

1 sudo apt install -y mosquitto mosquitto-clients
2 sudo systemctl enable mosquitto
3 sudo systemctl start mosquitto

```

9.4 Création du programme MQTT

```

1 cd ~/tp_iot
2 nano led_mqtt.py

```

Saisir :

```

1  #!/usr/bin/env python3
2  import os
3  import time
4  import json
5  from gpiozero import LED
6  import paho.mqtt.client as mqtt
7
8  # Configuration GPIO
9  LED_GPIO = int(os.getenv("LED_GPIO", "17"))
10
11 # Configuration MQTT (variables d'environnement ou valeurs par
    d faut)
12 CLIENT_ID = "rpi-led-publisher"
13 MQTT_HOST = os.getenv("MQTT_HOST", "test.mosquitto.org")
14 MQTT_PORT = int(os.getenv("MQTT_PORT", "1883"))
15 MQTT_TOPIC = os.getenv("MQTT_TOPIC", "rpi/led/status")
16
17 # Intervalle de clignotement (secondes)
18 INTERVALLE = float(os.getenv("INTERVALLE", "2"))
19
20 # Initialisation de la LED
21 led = LED(LED_GPIO)
22
23 # Callbacks MQTT
24 def on_connect(client, userdata, flags, rc):
25     if rc == 0:
26         print("Connect au broker MQTT:", MQTT_HOST)
27     else:
28         print("chec de connexion, code:", rc)
29
30 def on_publish(client, userdata, mid):
31     print(f" -> Message {mid} publi ")
32
33 # Client MQTT
34 client = mqtt.Client(client_id=CLIENT_ID)
35 client.on_connect = on_connect
36 client.on_publish = on_publish
37
38 print(f"Connexion au broker MQTT: {MQTT_HOST}:{MQTT_PORT}")
39 try:
40     client.connect(MQTT_HOST, MQTT_PORT, 60)
41     client.loop_start() # boucle rseau en arriere-plan
42 except Exception as e:
43     print(f"Erreur de connexion MQTT: {e}")
44     exit(1)
45
46 time.sleep(1.5) # attendre la connexion
47
48 print(f"\nPublications sur le topic: {MQTT_TOPIC}")
49 print("Ctrl+C pour arrter\n")
50

```



```

51 state = False
52 compteur = 0
53
54 try:
55     while True:
56         state = not state
57         compteur += 1
58
59         if state:
60             led.on()
61             etat_texte = "ON"
62         else:
63             led.off()
64             etat_texte = "OFF"
65
66         payload = {
67             "led": etat_texte,
68             "gpio": LED_GPIO,
69             "timestamp": int(time.time()),
70             "compteur": compteur
71         }
72         message_json = json.dumps(payload)
73         client.publish(MQTT_TOPIC, message_json, qos=0, retain=False)
74         print(f"[{compteur}] LED {etat_texte} - Publi : {
75             message_json}")
76
77         time.sleep(INTERVALLE)
78
79 except KeyboardInterrupt:
80     print("\nArr t...")
81
82 finally:
83     led.off()
84     client.loop_stop()
85     client.disconnect()
86     print("LED teinte - D connexion MQTT - Fin")

```

9.5 Exécution (broker public)

```
1 python3 led_mqtt.py
```

9.6 Vérification des messages MQTT

Sur un second terminal :

```
1 mosquitto_sub -h test.mosquitto.org -t "rpi/led/status" -v
```

Exemple de messages :

```

1 {"led": "ON", "gpio": 17, "timestamp": 1730241234, "compteur": 1}
2 {"led": "OFF", "gpio": 17, "timestamp": 1730241236, "compteur": 2}

```

9.7 Personnalisation (broker local)

```
1 export MQTT_HOST="192.168.1.100"  
2 export MQTT_TOPIC="mon_rpi/led"  
3 python3 led_mqtt.py
```

11. Dépannage

11.1 La LED ne s'allume pas

- Vérifier le câblage (GPIO 17 → résistance → anode LED → cathode → GND).
- Vérifier la polarité (patte longue = anode +).
- Test Python rapide :

```
1 from gpiozero import LED  
2 led = LED(17)  
3 led.on()  
4 led.off()
```

11.2 Erreur "Permission denied" pour GPIO

```
1 sudo usermod -aG gpio pi  
2 # Se d connecter/reconnecter
```

11.3 Erreur MQTT "Connection refused"

```
1 ping test.mosquitto.org  
2 telnet test.mosquitto.org 1883
```

11.4 SSH ne fonctionne pas

```
1 sudo systemctl status ssh  
2 sudo ufw status  
3 sudo raspi-config
```

Annexe : Pinout GPIO Raspberry Pi 4

